

Homework 4 – Refactoring Code

1. Encapsulation Smell (**Commit: 324da6d**):
 - a. **Before (lines 18–20):** target, moveCooldown, and frame were all public fields, allowing any class to write directly to them (e.g. `chaseState.frame = dx > 0 ? frameRight : frameLeft` at line 135). \
 - b. **After:** all three were made private with proper getters/setters added (lines 29–52). Call sites like `state.target` were updated to `state.getTarget()`, visible in both the integration test and unit test diffs.
2. `update()` Long Parameter List (**Commit: 324da6d**):
 - a. **Before:** We had 7 parameters: `Game_Map map, Player player, ChaseState chaseState, int frameLeft, int frameDown, int frameUp, int frameRight`.
 - b. **After:** the four int frame constants were grouped into a single `FrameSet` frames object, reducing it to 4 parameters.
3. `moveTowardPlayer()` Long Parameter List (**Commit: 324da6d**):
 - a. **Before:** We had 7 parameters, including the four frame ints separately.
 - b. **After:** replaced with `FrameSet` frames. Inside the method, `frameRight/frameLeft` became `frames.right/frames.left` (visible at lines 161/163).
4. Long Parameter List in `getFrame()` in `PlayerMovementHelper.java` (**Commit: c5f87b8**)
 - a. **Before:** `getFrame(Set<Integer> keysHeld, int currentFrame, int FRAME_LEFT, int FRAME_DOWN, int FRAME_UP, int FRAME_RIGHT)`. Two problems were visible here: 1. **CurrentFrame** was passed in but never read inside the method. 2. The four frame constants (`FRAME_LEFT`, etc.) were already defined on **Player** and were being passed in redundantly from outside.
 - b. **After:** `getFrame(Set<Integer> keysHeld)`. 1 parameter visible now. The method now references `Player.FRAME_UP`, `Player.FRAME_DOWN`, etc., directly. The call site in `Player.java` (line 48) went from passing 6 arguments down to 1.
5. Broken encapsulation in `Game_Entity.java` (**Commit: 44fea0c**)
 - a. **Before:** `position_x` and `position_y` were declared protected in `Game_Entity.java`, allowing subclasses and same-package classes to write to them directly (e.g. `this.position_x = x` in `Player.java` and `Enemy.java`, `item.position_x` in `GamePanel.java`), bypassing the `getX()/getY()/setX()/setY()` methods that already existed.
 - b. **After:** Both fields changed to private. All direct field accesses were replaced with the appropriate getters and setters across 7 files: `Player.java` (lines 28–29: `this.position_x = x` → `setX(x)`), `Enemy.java` (lines 22–23: same constructor fix), and `GamePanel.java` (lines 353, 360, 367: `item.position_x / item.position_y` → `item.getX() / item.getY()`), as well as `Item.java`, `Spawn_Main.java`, `Spawn_Bonus.java`, and `Spawn_Penalty.java`. The fact

that this change affected 7 files demonstrates how widespread the direct field access was throughout the codebase.

6. Poorly Structured Project / Feature Envy: **Game_Map** (Commit: a238144)

The **Game_Map** class was removed because it did not contain any significant logic of its own. Instead, it mostly acted as a wrapper that forwarded method calls to **Map_Builder**. Since the actual map-related behaviour was already implemented in **Map_Builder**, keeping **Game_Map** separated in this way created a feature envy smell, where one class depended too heavily on the functionality of another without contributing meaningful responsibility itself.

To address this, we removed **Game_Map** and refactored the parts of the project that depended on it so they could interact directly with **Map_Builder**. This change simplified the design and made the ownership of map functionality clearer. The affected classes included **Spawn_Main**, **Spawn_Bonus**, **Spawn_Penalty**, **Player**, **SecurityGuard**, **Pathfinder**, **Spawn_Helper**, and **Game**.

7. Poorly Structured Code in **Map_Builder** (Commit: 8743b63)

The constructor of **Map_Builder** originally contained too much implementation logic. In addition to initializing the object, it also directly handled the process of iterating through the map data, creating tiles, and storing them in the tile map. This made the constructor longer and less readable than necessary.

To improve the structure, I extracted the tile creation logic into a separate **createTileMap(String[] map)** method. After this refactoring, the constructor became more concise and focused only on initialization tasks such as setting the number of rows and columns, allocating the **tile_Map** array, and calling the helper method. This change improved readability and maintainability as it is easier to understand what the constructor does and easier to make changes to its methods.

8. Unjustified Use of Primitives in **Map_Builder** (Commit: 968aacc)

Originally, the map layout in **Map_Builder** was stored directly in the code as a large **String[] map**. While this worked functionally, it made the class less flexible, harder to maintain, and more difficult to modify without editing the source code.

To improve this, I moved the map layout out of the class and into an external JSON file. I then added a **loadMapFromJson(String resourcePath)** method that reads the file, parses the "map" array using **javax.json**, and returns the map data as a **String[]**. The constructor now loads the map from this resource file instead of storing the full layout directly in code.

This refactoring improved the separation of concerns by keeping the map data separate from the map-building logic. It also made the code easier to extend and maintain, since the layout can now be changed by editing the JSON resource rather than modifying Java source files.